UNIVERSIDADE FEDERAL DE UBERLÂNDIA
FACULDADE DE COMPUTAÇÃO
SEGURANÇA DA INFORMAÇÃO (GBC083)

MATHEUS DANTAS SANTANA
VANESSA DIAS FERREIRA

IMPLEMENTAÇÃO E ANÁLISE DE RAINBOW TABLES

1 INTRODUÇÃO E OBJETIVOS

Quando um sistema guarda senhas, ele geralmente não salva a senha em texto puro, mas sim o seu *hash* (calculado por uma função como o SHA-256). Essas funções são feitas para funcionar em apenas um sentido: é fácil calcular o hash a partir da senha, mas é muito difícil fazer o caminho inverso para descobrir qual senha gerou aquele hash.

Se um atacante tem apenas o hash e quer descobrir a senha, ele tem basicamente duas opções tradicionais:

- **Testar uma por uma (Força Bruta / Dicionário):** Vai pegando as palavras da lista, calcula o hash de cada uma e compara com o alvo. Não gasta quase nada de memória, mas se a lista for grande, demora muito tempo a cada nova tentativa.
- **Guardar tudo pronto (Tabela Pré-computada):** Calcula o hash de todas as palavras possíveis antes e salva tudo num arquivo. A busca fica instantânea, mas ocupa um espaço gigante em disco ou memória RAM.

A **Rainbow Table** é uma solução intermediária para equilibrar tempo e memória (*time-memory tradeoff*). Em vez de guardar todas as palavras e todos os hashes, ela cria **cadeias** aplicando repetidamente o hash e uma função de redução (que transforma o hash de volta em uma palavra da lista). No final, a tabela salva apenas a **primeira palavra** e a **última palavra** de cada cadeia.

Quando recebemos um hash para procurar, nós recalculamos alguns pedaços da cadeia até encontrar a senha. Assim, economizamos muito espaço em disco (pois descartamos todo o miolo da cadeia) gastando apenas um pouco de tempo para recalcular o que falta.

O objetivo deste trabalho é implementar uma Rainbow Table em Python para quebrar senhas baseadas na lista **Diceware** (com 7.776 palavras), comparando diferentes formatos de tabela (mais altas ou mais largas) e analisando a diferença entre usar várias reduções com cores ou uma redução fixa.

2 ESPECIFICAÇÃO DO HARDWARE E AMBIENTE DE TESTE

Como o tempo de cálculo depende diretamente da velocidade do computador, descrevemos abaixo a máquina utilizada para rodar todos os testes e experimentos:

Tabela 1 – Configuração do computador utilizado nos testes

Componente	Detalhes da Máquina
Processador (CPU)	AMD Ryzen 5 5500 (6 núcleos, 12 threads, clock base de 3.6 GHz)
Memória RAM	16 GB DDR4
Sistema Operacional	Windows 11 (64-bit)
Versão do Python	Python 3.13.7 (execução nativa, usando apenas a biblioteca padrão)
Alimentação e Condições	Computador ligado na tomada (AC), sem programas pesados rodando ao mesmo tempo
Lista de Palavras	Lista oficial Diceware com 7.776 palavras (iniciando em "a" e terminando em "@")

3 METODOLOGIA E TESTES ALEATÓRIOS

Para que a comparação entre os formatos fosse justa, todas as tabelas testadas foram configuradas para fazer exatamente o mesmo trabalho na construção: **8.192 passos de cálculo** (multiplicando o número de linhas m pelo comprimento da cadeia t). Como o dicionário tem 7.776 palavras, 8.192 passos são suficientes para cobrir teoricamente todo o conjunto de senhas.

Os testes foram organizados da seguinte forma:

1. **Sorteio dos 200 Alvos:** Sorteamos 200 senhas aleatórias da lista Diceware usando uma semente fixa (`2026`). Esse gerador de alvos foi totalmente separado do gerador da tabela, garantindo que o teste não fosse viciado.
2. **24 Cenários Testados:** Cruzamos **4 formatos** de tabela (512×16, 256×32, 128×64 e 64×128) × **3 sementes de tabela** (17, 43 e 91) × **2 modos de redução** (`cores` e `fixa`). Isso totalizou 24 cenários e 4.800 buscas cronometradas.
3. **Confirmação do Hash (Evitando Falsos Alarmes):** Encontrar o final da cadeia no índice não garante que a senha esteja nela (pode ter acontecido uma colisão no caminho). Por isso, a busca sempre refaz a cadeia a partir do início e só entrega a senha se o hash for exatamente igual ao procurado.

4 RESULTADOS: EFICÁCIA E EFICIÊNCIA

Medimos a **eficácia** pela quantidade de senhas recuperadas (de 0 a 200) e a **eficiência** pelo tempo médio de busca por consulta (em milissegundos) e pelo espaço que a tabela ocupa em disco (em bytes). A Tabela 2 traz as médias obtidas nas 3 sementes:

Tabela 2 – Resultados médios dos 24 cenários testados

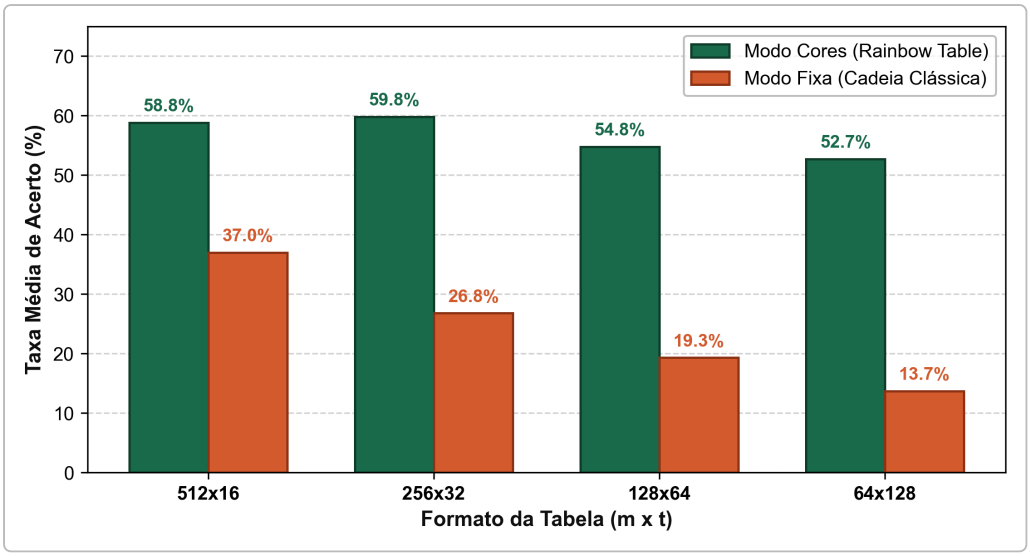
Modo	Formato (m × t)	Passos	Acertos Médios	Taxa (%)	Finais Únicos	Tam. Disco	Tempo Total Busca	Tempo no Sucesso	Tempo na Falha
cores	512 × 16	8.192	117,7 / 200	**58,8%**	328,0	8.430 B	**0,066 s**	0,19 ms	0,53 ms
cores	256 × 32	8.192	119,7 / 200	**59,8%**	160,7	4.249 B	**0,242 s**	0,68 ms	1,99 ms
cores	128 × 64	8.192	109,7 / 200	**54,8%**	80,7	2.101 B	**1,034 s**	2,61 ms	8,25 ms
cores	64 × 128	8.192	105,3 / 200	**52,7%**	40,3	1.058 B	**3,842 s**	7,38 ms	32,37 ms
fixa	512 × 16	8.192	74,0 / 200	37,0%	314,0	8.436 B	0,094 s	0,25 ms	0,60 ms
fixa	256 × 32	8.192	53,7 / 200	26,8%	147,0	4.175 B	0,448 s	1,11 ms	2,66 ms
fixa	128 × 64	8.192	38,7 / 200	19,3%	73,3	2.102 B	1,958 s	3,83 ms	11,23 ms
fixa	64 × 128	8.192	27,3 / 200	13,7%	38,0	1.067 B	8,170 s	9,64 ms	45,80 ms

4.1 Por que as Cores Ajudam Tanto?

No modo `fixa`, usamos sempre a mesma regra de redução. Se duas cadeias diferentes chegarem à mesma palavra no meio do caminho, todos os passos seguintes serão idênticos até o fim. Isso se chama **fusão de cadeias**: muito trabalho é repetido à toa e a tabela cobre poucas palavras diferentes. Por isso, quando a cadeia é mais longa (t = 128), a taxa de acerto no modo fixo despenca para apenas **13,7%**.

No modo `cores`, cada coluna tem uma cor (uma regra de redução diferente). Se duas cadeias coincidirem na mesma palavra em colunas diferentes, a próxima redução será diferente e elas se separam de novo. Elas só se fundem se baterem na mesma palavra e na mesma coluna. Graças a isso, a taxa de acerto se mantém alta em todos os formatos, ficando sempre entre **52,7% e 59,8%**.

Figura 1 – Taxa de acerto (%) Cores vs. Fixa

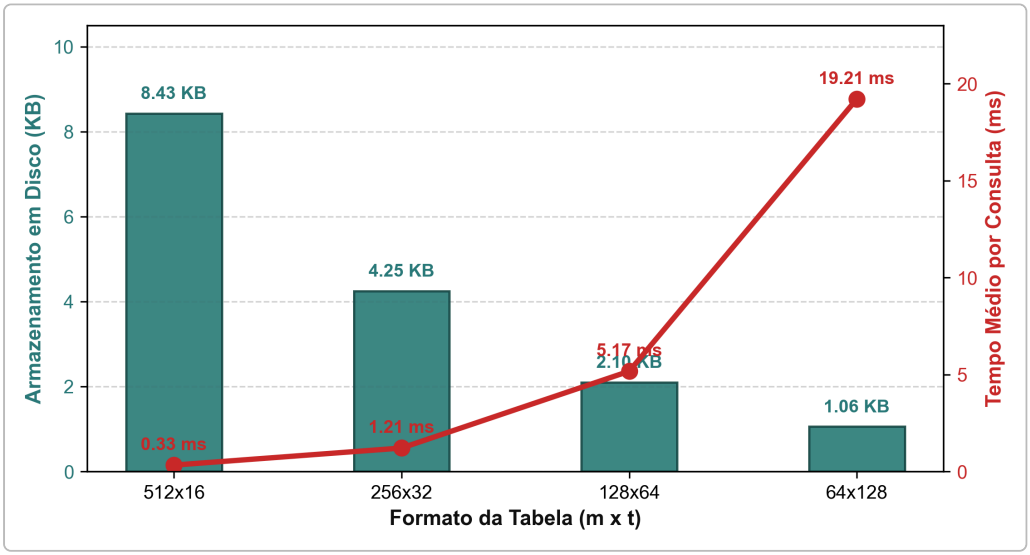


4.2 A Troca entre Espaço e Tempo de Busca

A Figura 2 mostra claramente o compromisso entre espaço e tempo:

- **Economia de Memória:** Na tabela 512×16 guardamos 512 pares (ocupa cerca de 8,4 KB). Na tabela 64×128 guardamos apenas 64 pares (ocupa 1,0 KB). Ou seja, reduzimos o espaço em disco em quase 88%.
- **Aumento no Tempo de Busca:** Em compensação, a busca na tabela 64×128 demorou 58 vezes mais que na tabela 512×16 (subiu de 0,066 s para 3,842 s no total dos 200 alvos). Isso acontece porque com t = 128 temos que testar muito mais posições e recalcular trechos bem maiores.
- **Sucesso vs. Falha:** Quando a busca tem sucesso, ela para assim que confirma o hash (leva de 0,19 ms a 7,38 ms). Quando a senha não está na tabela (falha), o algoritmo é obrigado a testar todas as hipóteses até o final antes de desistir (levando de 0,53 ms a 32,37 ms).

Figura 2 – Espaço em Disco (KB) vs. Tempo de Busca (ms)



5 CÓDIGO-FONTE E EXPLICAÇÃO DAS FUNÇÕES

O código foi dividido em três arquivos simples (`base.py`, `construcao.py` e `busca.py`). A seguir apresentamos o código de cada função e uma explicação simples do que ela faz.

5.1 Arquivo base.py

Quadro 1 – Função carregar_palavras (base.py)

```
def carregar_palavras(caminho: Path = LISTA) -> list[str]:
    codigo_para_palavra = {}
    for linha in caminho.read_text(encoding="utf-8").splitlines():
        campos = linha.split()
        if len(campos) != 2:
            continue
        codigo, palavra = campos
        if len(codigo) != 5 or any(digito not in "123456" for digito in codigo):
            continue
        if codigo in codigo_para_palavra:
            raise ValueError(f"Código Diceware duplicado: {codigo}")
        codigo_para_palavra[codigo] = palavra

    palavras = [codigo_para_palavra[codigo] for codigo in sorted(codigo_para_palavra)]

    if len(set(palavras)) != 7776:
        raise ValueError("A lista deve conter 7.776 códigos e palavras distintos")
    if palavras[0] != "a" or palavras[-1] != "@":
        raise ValueError("A lista deve começar em 'a' e terminar em '@'")

    return palavras
```

O que faz: Abre o arquivo `diceware.wordlist.asc`, ignora as linhas extras (como avisos de assinatura PGP) e confere se cada linha tem o código de 5 números (de 1 a 6) e a palavra. Guarda tudo num dicionário para não aceitar códigos repetidos e devolve a lista com exatamente as 7.776 palavras organizadas em ordem.

Quadro 2 – Função calcular_hash (base.py)

```
def calcular_hash(senha: str) -> str:
    return hashlib.sha256(b"GBC083" + senha.encode("utf-8")).hexdigest()
```

O que faz: Calcula o hash SHA-256 da senha adicionando o prefixo `b"GBC083"` antes dela, exatamente como o professor pediu no slide 47 da aula. Retorna o hash em formato de texto hexadecimal com 64 caracteres.

Quadro 3 – Função reduzir (base.py)

```
def reduzir(h: str, coluna: int, palavras: list[str], modo: str = "cores") -> str:
    if coluna < 0 or not palavras or modo not in {"cores", "fixa"}:
        raise ValueError("Exigidos coluna >= 0, lista não vazia e modo cores ou fixa")

    cor = coluna if modo == "cores" else 0

    mistura = hashlib.sha256(f"{cor}:{h}".encode("utf-8")).hexdigest()
    posicao = int(mistura, 16) % len(palavras)

    return palavras[posicao]
```

O que faz: Transforma um hash de volta em uma palavra da lista. Se o modo for `cores`, usa o número da coluna no texto (`f"{cor}:{h}"`), fazendo com que colunas diferentes dêem resultados diferentes. Se for `fixa`, usa sempre 0. Depois calcula o SHA-256 dessa mistura e pega o resto da divisão por 7.776 para escolher o índice da palavra.

5.2 Arquivo construcao.py

Quadro 4 – Função fim_da_cadeia (construcao.py)

```
def fim_da_cadeia(inicio: str, palavras: list[str], largura: int, modo: str = "cores") -> str:
    if largura < 1:
        raise ValueError("A largura deve ser pelo menos 1")

    atual = inicio
    for coluna in range(largura):
        h = calcular_hash(atual)
        atual = reduzir(h, coluna, palavras, modo)

    return atual
```

O que faz: Monta uma cadeia completa. Começa na palavra inicial, calcula o hash dela, reduz para achar a próxima palavra e repete isso *largura* vezes. No final, devolve apenas a última palavra encontrada (o miolo é descartado).

Quadro 5 – Função construir (construcao.py)

```
def construir(palavras: list[str], altura: int, largura: int, semente: int, modo: str = "cores"
    if not 1 <= altura <= len(palavras) or largura < 1:
        raise ValueError("Exigidos 1 <= altura <= tamanho da lista e largura >= 1")

    inicios = random.Random(semente).sample(palavras, altura)
    pares = []

    for inicio in inicios:
        fim = fim_da_cadeia(inicio, palavras, largura, modo)
        pares.append((inicio, fim))

    return pares
```

O que faz: Constrói a tabela inteira. Usa a semente para sortear *altura* palavras iniciais sem repetição. Para cada uma delas, chama a função `fim_da_cadeia` e salva na lista o par (início, fim).

5.3 Arquivo busca.py

Quadro 6 – Função indexar (busca.py)

```
def indexar(pares: list[tuple[str, str]]) -> Indice:
    indice: Indice = {}
    for inicio, fim in pares:
        if fim not in indice:
            indice[fim] = []
        indice[fim].append(inicio)
    return indice
```

O que faz: Organiza os pares num dicionário onde a chave é a palavra final e o valor é a lista de inícios que terminam nela. Se dois inícios diferentes chegarem ao mesmo final, ambos são guardados na lista. Isso permite consultar qualquer final de forma instantânea.

Quadro 7 – Função buscar (busca.py)

```
def buscar(alvo: str, palavras: list[str], indice: Indice, largura: int,
           modo: str = "cores") -> str | None:
    if largura < 1:
        raise ValueError("A largura deve ser pelo menos 1")

    for k in range(largura - 1, -1, -1):
        hash_atual = alvo
        for j in range(k, largura):
            palavra = reduzir(hash_atual, j, palavras, modo)
            if j < largura - 1:
                hash_atual = calcular_hash(palavra)

        final_candidato = palavra
        for inicio in indice.get(final_candidato, []):
            atual = inicio
            for coluna in range(largura):
                h = calcular_hash(atual)
                if h == alvo:
                    return atual
                atual = reduzir(h, coluna, palavras, modo)

    return None
```

O que faz: Recebe o hash que queremos encontrar e tenta adivinhar em qual coluna ele foi gerado (testando de trás para frente, da última coluna até a primeira). Em cada hipótese, caminha até o final da cadeia e vê se aquele final existe no índice. Se existir, pega o início correspondente e refaz a cadeia passo a passo. Se encontrar o hash procurado, devolve a palavra na hora; se testar tudo e não achar, devolve `None`.